

Task 8: Session 8 (AIML) - Assignment Questions

Name: Mihir Sopan Patil

Domain: AIML

College & Branch Name: Zeal Polytechnic, Diploma In AIML

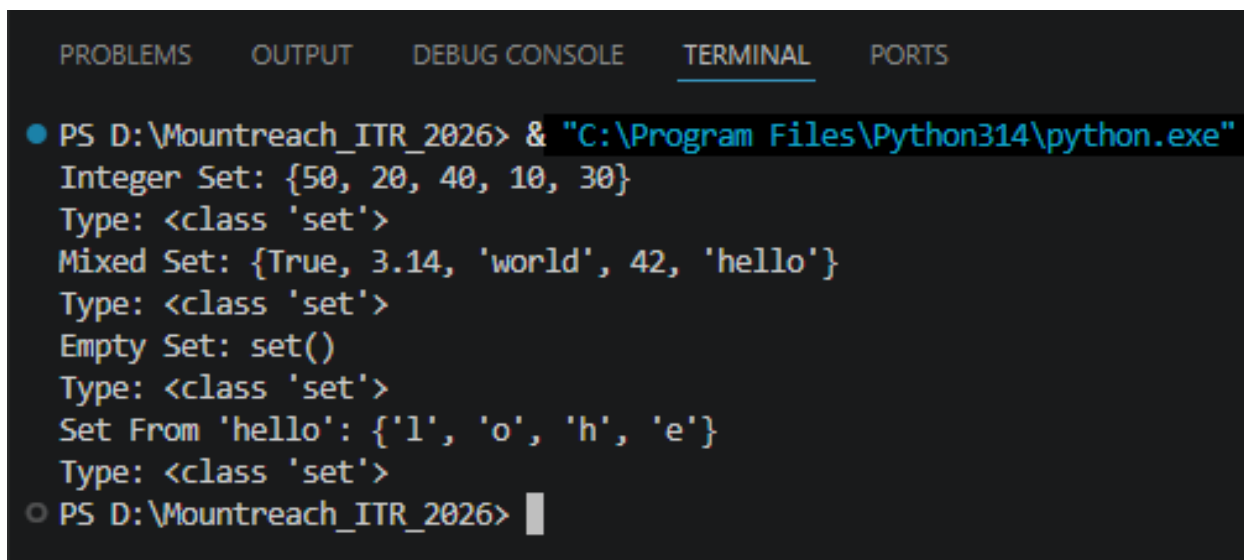
1. Creating Sets

Write a program to create the following sets:

- A set with 5 integers.
- A set with mixed data types (integer, string, float).
- An empty set using the correct method.
- A set from the string "hello" using the set() constructor.

Print each set and its type. Add comments explaining why sets automatically remove duplicates.

Output:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
● PS D:\Mountreach_ITR_2026> & "C:\Program Files\Python314\python.exe"
Integer Set: {50, 20, 40, 10, 30}
Type: <class 'set'>
Mixed Set: {True, 3.14, 'world', 42, 'hello'}
Type: <class 'set'>
Empty Set: set()
Type: <class 'set'>
Set From 'hello': {'l', 'o', 'h', 'e'}
Type: <class 'set'>
○ PS D:\Mountreach_ITR_2026> |
```

Program Code:

```
# A Set With 5 Integers
int_set = {10, 20, 30, 40, 50}
print("Integer Set:", int_set)
print("Type:", type(int_set))

# A Set With Mixed Data Types
mixed_set = {42, "hello", 3.14, True, "world"}
print("Mixed Set:", mixed_set)
print("Type:", type(mixed_set))

# An Empty Set Must Use set(), Not {} (That Creates An Empty Dict)
empty_set = set()
print("Empty Set:", empty_set)
print("Type:", type(empty_set))

# A Set From The String "hello" Using set() Constructor
# Sets Store Only Unique Elements, So Duplicate Letters Are Removed
string_set = set("hello")
print("Set From 'hello':", string_set)
print("Type:", type(string_set))

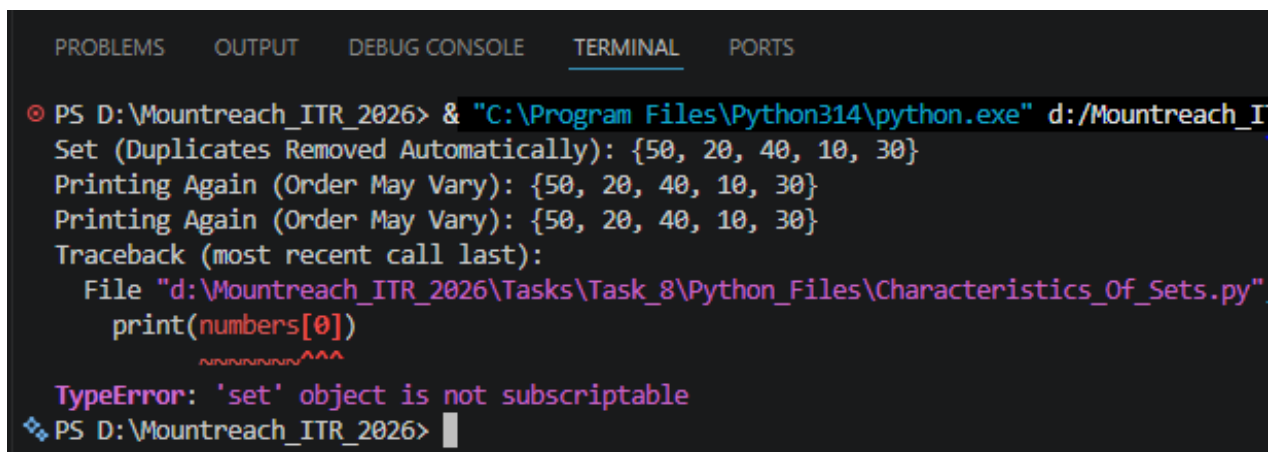
# Why Sets Remove Duplicates:
# Sets Are Implemented Using Hash Tables.
# Each Element Is Hashed & Stored At A Unique Position.
# If Two Elements Have The Same Hash (i.e., They Are Equal), Only One Is Kept.
```

2. Characteristics Of Sets

Create a set: numbers = {10, 20, 30, 20, 40, 10, 50}

- Print the set and observe the output.
- Demonstrate that sets are unordered by printing the set multiple times.
- Try to access the first element using indexing (numbers[0]) and explain the error in comments.

Output:



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS D:\Mountreach_ITR_2026> & "C:\Program Files\Python314\python.exe" d:/Mountreach_I
Set (Duplicates Removed Automatically): {50, 20, 40, 10, 30}
Printing Again (Order May Vary): {50, 20, 40, 10, 30}
Printing Again (Order May Vary): {50, 20, 40, 10, 30}
Traceback (most recent call last):
  File "d:\Mountreach_ITR_2026\Tasks\Task_8\Python_Files\Characteristics_Of_Sets.py",
    print(numbers[0])
          ~~~~~^~~~
TypeError: 'set' object is not subscriptable
PS D:\Mountreach_ITR_2026> 
```

Program Code:

```
numbers = {10, 20, 30, 20, 40, 10, 50}

print("Set (Duplicates Removed Automatically):", numbers)
print("Printing Again (Order May Vary):", numbers)
print("Printing Again (Order May Vary):", numbers)

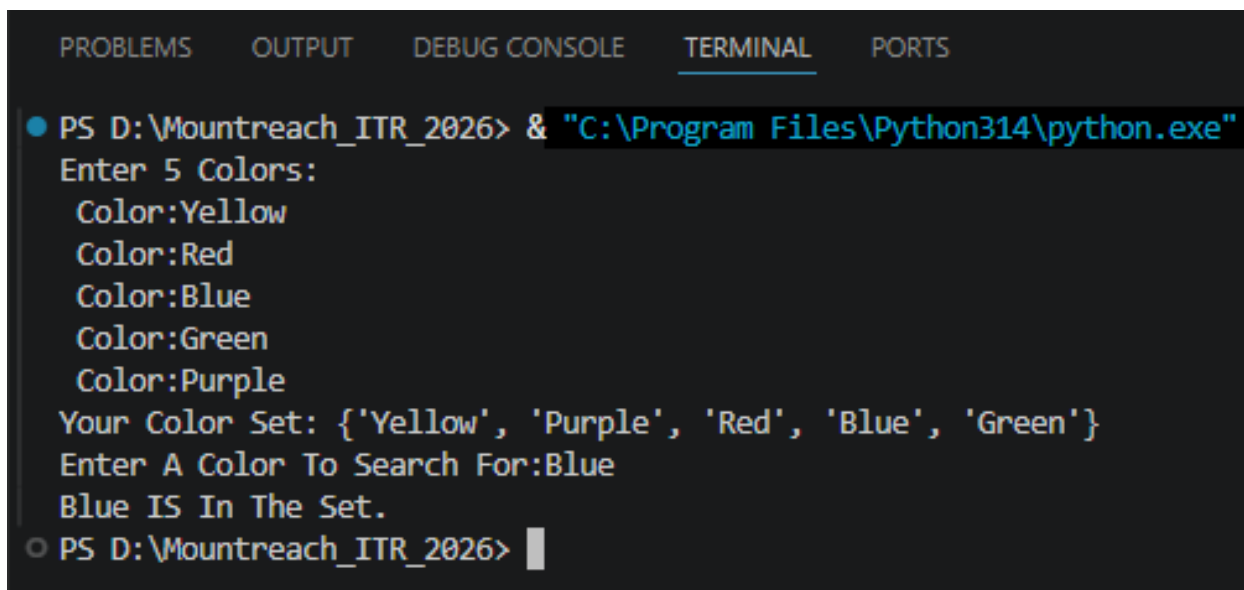
# Python Does Not Guarantee Any Apecific Order When Iterating Or Printing A Set.
print(numbers[0])
# Trying Indexing - This Will Raise A TypeError:
# Error: 'set' Object Is Not Subscriptable
# Sets Do Not Support Indexing Or Slicing Because They Are Unordered.
# There Is No Concept Of A "first" Element In A Set.
print("Indexing (numbers[0]) Is Not Supported On Sets - Sets Are Unordered.")
```

3. Membership Testing

Write a program that:

- Takes a set of 5 colors as input from the user.
- Asks the user to enter a color name.
- Checks whether the color is present in the set using in and not in.
- Prints appropriate messages.

Output:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
● PS D:\Mountreach_ITR_2026> & "C:\Program Files\Python314\python.exe"
Enter 5 Colors:
Color:Yellow
Color:Red
Color:Blue
Color:Green
Color:Purple
Your Color Set: {'Yellow', 'Purple', 'Red', 'Blue', 'Green'}
Enter A Color To Search For:Blue
Blue IS In The Set.
○ PS D:\Mountreach_ITR_2026> |
```

Program Code:

```
colors = set()
print("Enter 5 Colors:")
for i in range(5):
    color = input(" Color:")
    colors.add(color)

print("Your Color Set:", colors)
search = input("Enter A Color To Search For:")

if search in colors:
    print(search, "IS In The Set.")
else:
    print(search, "Is NOT In The Set.")

if search not in colors:
    print("Confirmed:", search, "Is Not Present.")
```

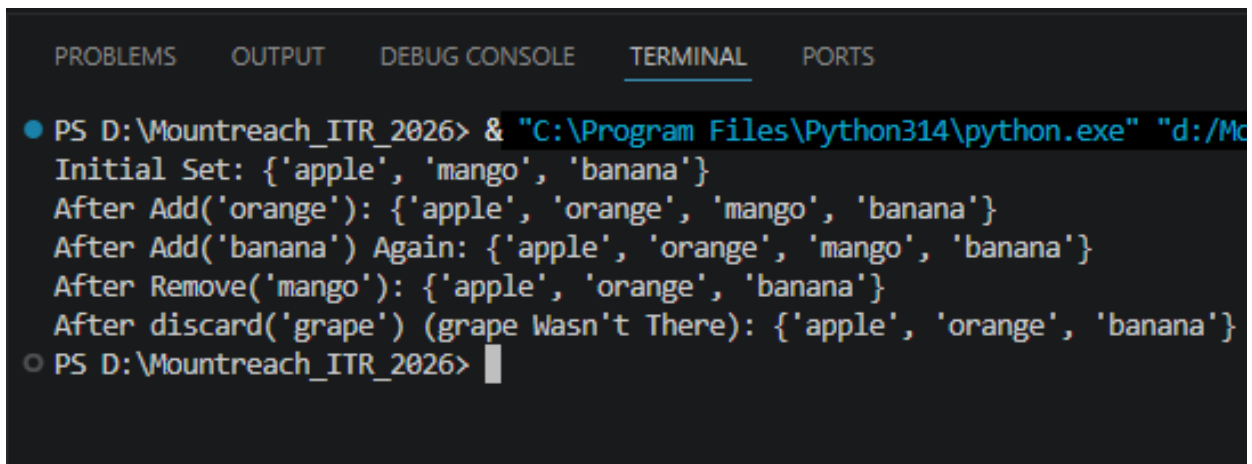
4. add(), remove(), discard()

Create a set: fruits = {'apple', 'banana', 'mango'}

Perform the following operations and print the set after each step:

- Add 'orange' using add().
- Add 'banana' again (observe the result).
- Remove 'mango' using remove().
- Try to remove 'grape' using discard() (no error should occur).

Output:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
● PS D:\Mountreach_ITR_2026> & "C:\Program Files\Python314\python.exe" "d:/M
Initial Set: {'apple', 'mango', 'banana'}
After Add('orange'): {'apple', 'orange', 'mango', 'banana'}
After Add('banana') Again: {'apple', 'orange', 'mango', 'banana'}
After Remove('mango'): {'apple', 'orange', 'banana'}
After discard('grape') (grape Wasn't There): {'apple', 'orange', 'banana'}
○ PS D:\Mountreach_ITR_2026> █
```

Program Code:

```
fruits = {'apple', 'banana', 'mango'}
print("Initial Set:", fruits)

# Add 'orange'
fruits.add('orange')
print("After Add('orange'):", fruits)

# Add 'banana' Again - No Duplicate, No Error, Set Unchanged
fruits.add('banana')
print("After Add('banana') Again:", fruits) # banana Already Exists

# Remove 'mango' - remove() Raises KeyError If Element Doesn't Exist
fruits.remove('mango')
print("After Remove('mango'):", fruits)

# discard 'grape' - discard() Does Not Raise An Error If Missing
fruits.discard('grape')
print("After discard('grape') (grape Wasn't There):", fruits)
```

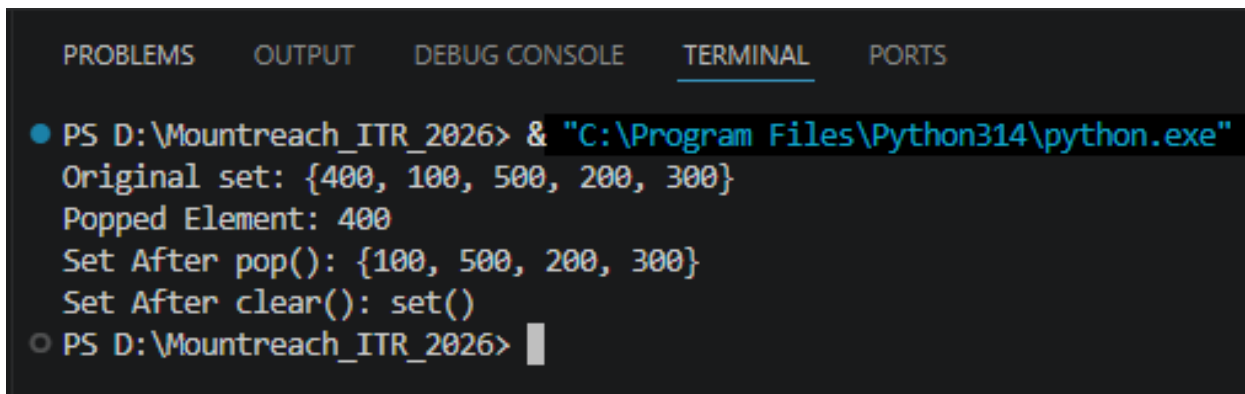
5. pop() & clear()

Create a set: `s = {100, 200, 300, 400, 500}`

- Use `pop()` to remove and print one element.
- Print the set after popping.
- Clear the entire set using `clear()`.
- Print the final empty set.

Add comments explaining the difference between `remove()`, `discard()`, and `pop()`.

Output:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
● PS D:\Mountreach_ITR_2026> & "C:\Program Files\Python314\python.exe"
Original set: {400, 100, 500, 200, 300}
Popped Element: 400
Set After pop(): {100, 500, 200, 300}
Set After clear(): set()
○ PS D:\Mountreach_ITR_2026> █
```

Program Code:

```
s = {100, 200, 300, 400, 500}
print("Original set:", s)

# pop() Removes & Returns An Arbitrary Element (Unpredictable Which One)
popped = s.pop()
print("Popped Element:", popped)
print("Set After pop():", s)

# clear() Removes All Elements From The Set
s.clear()
print("Set After clear():", s)

# Differences:
# remove(x) -> Removes Element x; Raises KeyError If x Is Not Found
# discard(x) -> Removes Element x; Does Nothing If x Is Not Found
# pop() -> Removes & Returns A Random Element; Raises KeyError If Set Is Empty
```

6. update() & copy()

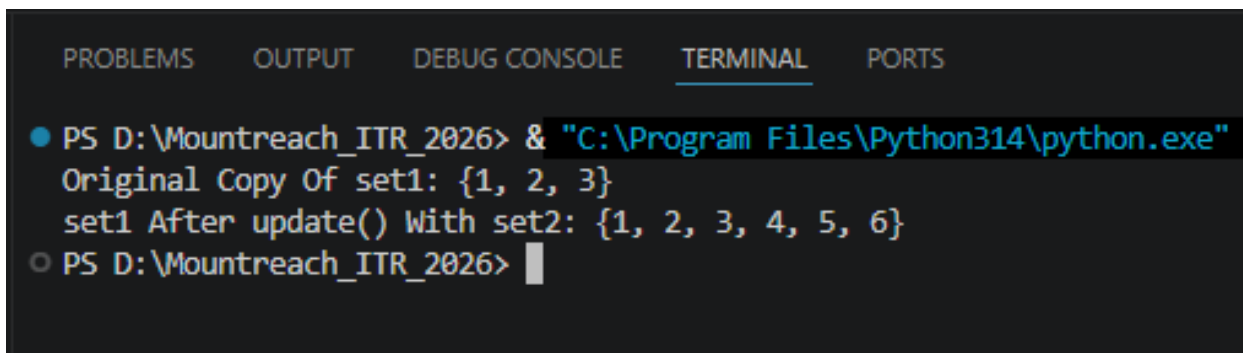
Create two sets:

```
set1 = {1, 2, 3}
```

```
set2 = {3, 4, 5, 6}
```

- Make a copy of set1.
- Update set1 with elements from set2 using update().
- Print both the original copy and the updated set.

Output:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

● PS D:\Mountreach_ITR_2026> & "C:\Program Files\Python314\python.exe"
  Original Copy Of set1: {1, 2, 3}
  set1 After update() With set2: {1, 2, 3, 4, 5, 6}
○ PS D:\Mountreach_ITR_2026> █
```

Program Code:

```
set1 = {1, 2, 3}
set2 = {3, 4, 5, 6}

# copy() Creates A Shallow Copy - Changes To set1 Won't Affect set1_copy
set1_copy = set1.copy()

# update() Adds All Elements From set2 Into set1 (In-Place Union)
set1.update(set2)

print("Original Copy Of set1:", set1_copy)
print("set1 After update() With set2:", set1)
```

7. Union & Intersection

Create two sets:

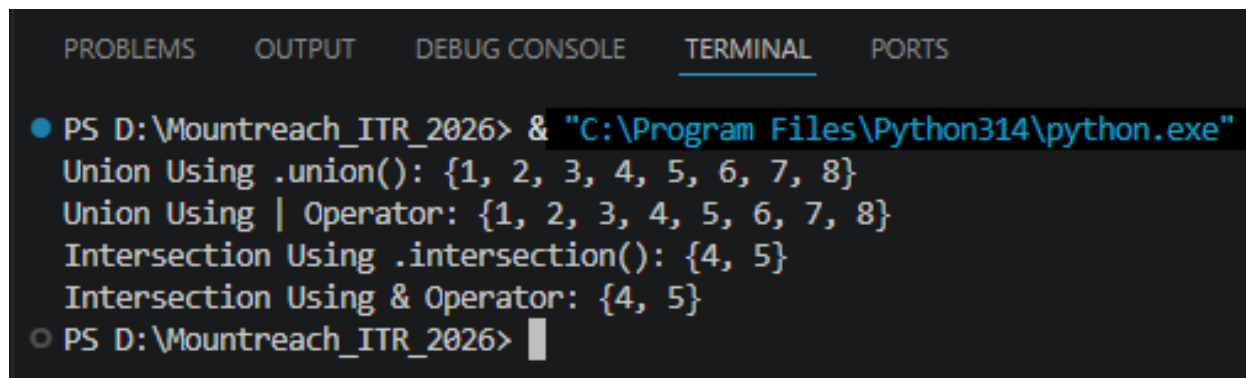
A = {1, 2, 3, 4, 5}

B = {4, 5, 6, 7, 8}

- Print the union of A and B (using both `.union()` and operator).
- Print the intersection of A and B (using both `.intersection()` and `&` operator).

Add comments explaining what each operation does.

Output:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
● PS D:\Mountreach_ITR_2026> & "C:\Program Files\Python314\python.exe"
  Union Using .union(): {1, 2, 3, 4, 5, 6, 7, 8}
  Union Using | Operator: {1, 2, 3, 4, 5, 6, 7, 8}
  Intersection Using .intersection(): {4, 5}
  Intersection Using & Operator: {4, 5}
○ PS D:\Mountreach_ITR_2026> █
```

Program Code:

```
A = {1, 2, 3, 4, 5}
B = {4, 5, 6, 7, 8}

# Union: All Elements From Both Sets (No Duplicates)
print("Union Using .union():", A.union(B))
print("Union Using | Operator:", A | B)

# Intersection: Only Elements That Appear In Both Sets
print("Intersection Using .intersection():", A.intersection(B))
print("Intersection Using & Operator:", A & B)

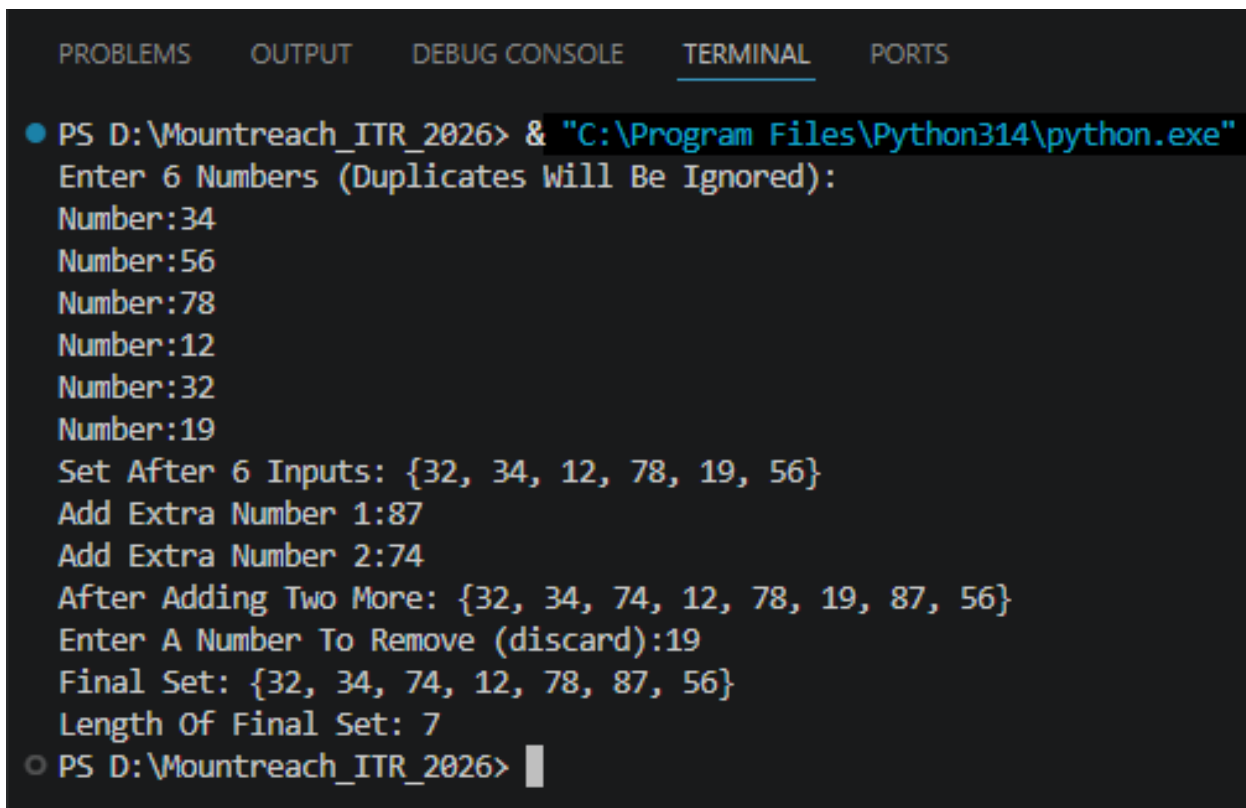
# Union Gives Every Unique Element Across Both Sets.
# Intersection Gives Only The Common Elements Shared By Both Sets.
```


8. Combined Methods

Write a program that:

- Takes 6 numbers as input from the user and stores them in a set (duplicates should be removed automatically).
- Adds two more numbers using add().
- Removes one number safely using discard().
- Prints the final set and its length using len().

Output:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
● PS D:\Mountreach_ITR_2026> & "C:\Program Files\Python314\python.exe"
Enter 6 Numbers (Duplicates Will Be Ignored):
Number:34
Number:56
Number:78
Number:12
Number:32
Number:19
Set After 6 Inputs: {32, 34, 12, 78, 19, 56}
Add Extra Number 1:87
Add Extra Number 2:74
After Adding Two More: {32, 34, 74, 12, 78, 19, 87, 56}
Enter A Number To Remove (discard):19
Final Set: {32, 34, 74, 12, 78, 87, 56}
Length Of Final Set: 7
○ PS D:\Mountreach_ITR_2026> █
```

Program Code:

```
user_set = set()

print("Enter 6 Numbers (Duplicates Will Be Ignored):")
for i in range(6):
    num = int(input("Number:"))
    user_set.add(num)

print("Set After 6 Inputs:", user_set)
```

```
# Add Two More Numbers
extra1 = int(input("Add Extra Number 1:"))
user_set.add(extra1)
extra2 = int(input("Add Extra Number 2:"))
user_set.add(extra2)
print("After Adding Two More:", user_set)

# Safely Remove One Number
remove_val = int(input("Enter A Number To Remove (discard):"))
user_set.discard(remove_val) # No Error Even If Not Present

print("Final Set:", user_set)
print("Length Of Final Set:", len(user_set))
```

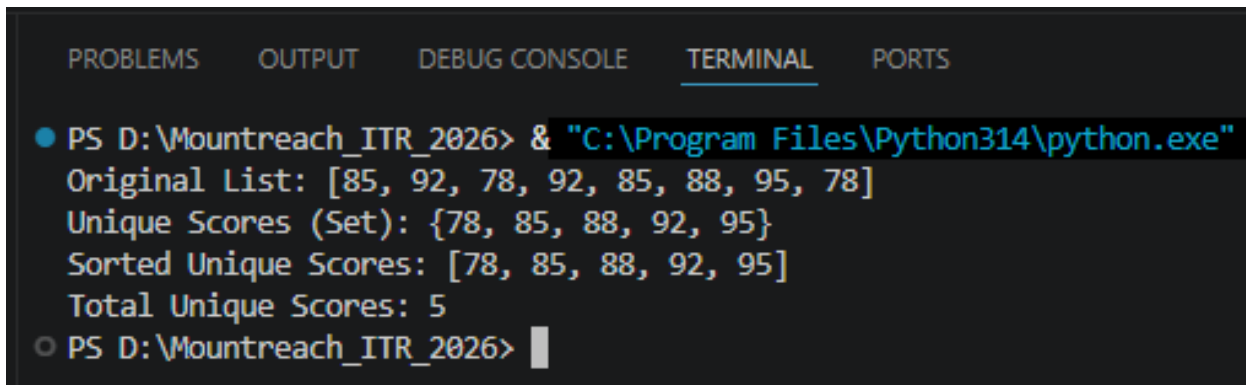
9. Practical Application

Write a program to remove duplicate marks from a list of student scores using a set:

```
scores = [85, 92, 78, 92, 85, 88, 95, 78]
```

- Convert the list to a set.
- Print the unique scores.
- Convert the set back to a sorted list and print it.
- Print the total number of unique scores.

Output:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
● PS D:\Mountreach_ITR_2026> & "C:\Program Files\Python314\python.exe"
Original List: [85, 92, 78, 92, 85, 88, 95, 78]
Unique Scores (Set): {78, 85, 88, 92, 95}
Sorted Unique Scores: [78, 85, 88, 92, 95]
Total Unique Scores: 5
○ PS D:\Mountreach_ITR_2026> █
```

Program Code:

```
scores = [85, 92, 78, 92, 85, 88, 95, 78]
print("Original List:", scores)

# Convert List To Set - Duplicates Removed Automatically
unique_scores = set(scores)
print("Unique Scores (Set):", unique_scores)

# Convert Back To A Sorted List
sorted_scores = sorted(list(unique_scores))
print("Sorted Unique Scores:", sorted_scores)
print("Total Unique Scores:", len(sorted_scores))
```

10. Mini Project - Combined Concepts

Create a **Unique Item Collector** program using sets:

- Repeatedly ask the user to enter items (e.g., favorite fruits, colors, or numbers).
- Store all entered items in a set (duplicates should be ignored).
- Provide a menu with options:
 - a) Add item
 - b) Remove item (use discard)
 - c) Show all unique items
 - d) Check if an item exists
 - e) Clear all items
 - f) Exit

Use a while loop for the menu and appropriate set methods. Add meaningful comments.

Output:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

● PS D:\Mountreach_ITR_2026> & "C:\Program Files\Python314\python.exe"

*** Menu ***
1. Add Item
2. Remove Item
3. Show All Unique Items
4. Check If An Item Exists
5. Clear All Items
6. Exit
Choose An Option (1-6):1
Enter Item To Add:Mihir
Mihir added. Current items: {'Mihir'}

*** Menu ***
1. Add Item
2. Remove Item
3. Show All Unique Items
4. Check If An Item Exists
5. Clear All Items
6. Exit
Choose An Option (1-6):6
Goodbye!
○ PS D:\Mountreach_ITR_2026> 
```

Program Code:

```
items = set() # Main Collection

while True:
    print("\n*** Menu ***")
    print("1. Add Item")
    print("2. Remove Item")
    print("3. Show All Unique Items")
    print("4. Check If An Item Exists")
    print("5. Clear All Items")
    print("6. Exit")

    choice = int(input("Choose An Option (1-6):"))

    if choice == 1:
        item = input("Enter Item To Add:")
        items.add(item) # add() Ignores The Item If It Already Exists
        print(item, "added. Current items:", items)

    elif choice == 2:
        item = input("Enter Item To Remove:")
        items.discard(item) # discard() Won't Raise Error If Item Is Missing
        print(item, "Removed (If It Existed). Current Items:", items)

    elif choice == 3:
        if items:
            print("All Unique Items:", items)
        else:
            print("The Collection Is Empty.")

    elif choice == 4:
        item = input("Enter Item To Check:")
        if item in items:
            print(item, "Exists In The Collection.")
        else:
            print(item, "Does Not Exist In The Collection.")

    elif choice == 5:
        items.clear() # Removes All Elements; Set Becomes Empty
        print("All Items Cleared.", items)

    elif choice == 6:
        print("Goodbye!")
        break

    else:
        print("Invalid Choice. Please Enter A Number Between 1 & 6.")
```